BOOTCAMP II · ENTREGA INICIAL · FASE 1

Planejamento Lógico TaskTracker

Documento de arquitetura e receita lógica para orientar a implementação em Python.

Identificação acadêmica — preencher antes da entrega
Nome completo: _____
RA / matrícula: _____ Curso/turma: _____
Polo: _____ E-mail institucional: _____

Sobre este documento: este planejamento foi elaborado agora para apoiar as Fases 1 e 2. A Fase 1 não havia sido realizada anteriormente; este arquivo não afirma entrega, aprovação ou avaliação acadêmica prévia.

1. Identificação e entendimento do problema

Contexto e problema

Na rotina de estudos, trabalho e atividades pessoais, é comum acumular tarefas sem um lugar simples para registrar o que precisa ser feito. Anotações espalhadas podem dificultar a consulta, a definição de prioridades e o acompanhamento de prazos. Uma tarefa esquecida ou sem informação clara pode atrasar outras atividades.

Objetivo e solução proposta

O TaskTracker será um programa de linha de comando para cadastrar tarefas e consultar a lista durante sua execução. Para cada tarefa, o usuário informará título, descrição, prioridade e data limite. O sistema verificará se o título foi preenchido e se a prioridade pertence às opções permitidas. Cada registro começará automaticamente com o status “Pendente”.

Público e limites

O protótipo poderá ser usado por estudantes e por qualquer pessoa que queira organizar tarefas de forma direta no terminal. Na Fase 2, os dados ficarão em memória enquanto o programa estiver aberto. Não haverá banco de dados, persistência, edição ou exclusão. O objetivo é praticar decomposição do problema, lógica, Python e versionamento.

2. Decomposição do problema, dados e regras

Entradas

Dado	Uso
Título	Identifica a tarefa. É obrigatório; será armazenado sem espaços extras nas extremidades.
Descrição	Texto livre para detalhar a tarefa; poderá ficar vazio.
Prioridade	Alta, Média/Media ou Baixa; será padronizada ao armazenar.
Data limite	Texto informado para registrar a data planejada, sem validação avançada.

Saídas

- Confirmação quando uma tarefa for cadastrada.
- Mensagens de erro e novo pedido de entrada se título ou prioridade forem inválidos.
- Lista organizada com todas as propriedades das tarefas.
- Mensagem “Nenhuma tarefa cadastrada no momento.” quando a lista estiver vazia.

Regras

Regra	Comportamento
Título obrigatório	Rejeitar texto vazio ou composto somente de espaços; repetir a solicitação.
Prioridade restrita	Aceitar Alta, Média, Media ou Baixa sem diferenciar maiúsculas e minúsculas; repetir em caso de erro.
Descrição e data	Descrição livre e data limite em texto, sem regra adicional de formato.
Status inicial	Definir automaticamente “Pendente” no cadastro.
Estrutura	Guardar cada tarefa em um dicionário e os dicionários em uma lista.

O dicionário terá as chaves `titulo`, `descricao`, `prioridade`, `data_limite` e `status`. A lista será criada ao iniciar; os dados não serão preservados depois do encerramento.

3. Receita lógica — algoritmo em português

1. Iniciar o programa.
2. Criar uma lista vazia chamada tarefas.
3. Exibir o menu: cadastrar tarefa, visualizar tarefas ou sair.
4. Ler a opção escolhida.
5. Se a opção for cadastrar, solicitar o título e remover espaços das extremidades.
6. Enquanto o título estiver vazio, exibir erro e pedir outro título.
7. Solicitar a descrição.
8. Solicitar a prioridade e comparar sem diferenciar maiúsculas e minúsculas com Alta, Média, Media e Baixa.
9. Enquanto a prioridade não for aceita, exibir erro e perguntar novamente. Armazenar a prioridade padronizada.
10. Solicitar a data limite e armazená-la como texto.
11. Criar um dicionário com título, descrição, prioridade, data limite e status “Pendente”.
12. Adicionar o dicionário à lista e confirmar o cadastro.
13. Se a opção for visualizar, verificar se a lista está vazia. Se estiver, avisar; caso contrário, percorrer a lista e exibir todas as propriedades.
14. Se a opção não for cadastrar, visualizar nem sair, informar que ela é inválida.
15. Depois de cadastrar, listar ou mostrar erro, retornar ao menu. Encerrar apenas quando o usuário escolher sair.

Estrutura prevista para cada registro

```
tarefa = {
    "titulo": titulo,
    "descricao": descricao,
    "prioridade": prioridade,
    "data_limite": data_limite,
    "status": "Pendente"
}
```

4. Mapeamento arquitetural — Git e GitHub

Git é um sistema local de controle de versão que registra mudanças do projeto. Cada commit guarda uma etapa coerente, com uma mensagem que explica seu propósito, permitindo acompanhar a evolução do trabalho.

GitHub hospeda o repositório Git remotamente para publicar código e documentos. A branch principal planejada é `main`; o envio será feito após autorização do estudante.

Estrutura planejada para a Fase 2

```
tasktracker-projeto/
├── README.md
├── .gitignore
├── docs/
│   └── planejamento_logico.pdf
└── src/
    └── main.py
```

O README apresenta finalidade e instruções; o .gitignore ignora arquivos temporários; o PDF guarda este planejamento; e `src/main.py` conterá a aplicação CLI em Python.

Etapas de versionamento previstas

1. Registrar estrutura, README inicial, .gitignore e planejamento.
2. Adicionar menu e estrutura inicial do programa.
3. Adicionar cadastro, validações e listagem.
4. Revisar implementação e documentação.

Após os commits locais, serão conferidos branch, histórico e estado do repositório. Com autorização, será criado um repositório público e enviada a branch `main`. Os PDFs de referência e credenciais ficam fora do repositório.

5. Link da apresentação em vídeo

Link do vídeo: PENDENTE — preencher após a gravação

O enunciado prevê um vídeo de 5 minutos no YouTube, horizontal (16:9), com o estudante em câmera e áudio claro. A visibilidade deve ser Público ou Não Listado. Shorts/vertical, vídeo privado e gravação apenas da tela sem o rosto não atendem às instruções. A Fase 2 também requer voz própria.

Apresentação prevista

O estudante poderá apresentar sua identificação acadêmica, explicar o problema, mostrar este documento, percorrer o algoritmo e comentar a estrutura planejada para GitHub. O vídeo ainda não foi gravado nem publicado.

Continuidade entre fases

Fase	Continuidade
Fase 1 — Planejamento lógico (10 pontos)	Definir problema, dados, regras, algoritmo, estrutura Git/GitHub e preparar apresentação.
Fase 2 — Entrega intermediária (15 pontos)	Usar o planejamento, criar repositório, implementar CLI em Python, validar regras, registrar commits e demonstrar o código.
Fase 3 — Entrega final (25 pontos)	Etapa futura, fora do escopo atual. Docker e deploy não serão implementados.

Este documento orienta o desenvolvimento. Entrega e avaliação dependem da submissão e dos critérios do curso.